Relaciones entre tablas en JPA

¿Qué es una relación en JPA?

En una base de datos relacional, los datos suelen estar repartidos en **varias tablas** que se relacionan entre sí mediante **claves primarias y foráneas**.
En **JPA**, estas relaciones se representan **como asociaciones entre clases Java**.

Cada relación entre tablas se define mediante **anotaciones** en las clases de entidad, que indican cómo están conectadas y cómo deben comportarse al guardar o recuperar los datos.

Tipos de relaciones en JPA

En JPA existen **cuatro tipos principales de relaciones**, que se corresponden con las relaciones habituales en bases de datos relacionales:

Tipo de relación	Descripción	Ejemplo típico
`@OneToOne`	Una entidad se asocia con **una única instancia**de otra entidad.	Un alumno tiene un único perfil.
`@OneToMany`	Una entidad se asocia con **varias instancias** de otra entidad.	Un alumno tiene muchas notas.
`@ManyToOne`	Muchas entidades se asocian con **una sola instancia** de otra entidad.	Muchas notas pertenecen a un alumno.
`@ManyToMany`	Varias entidades se asocian con **varias instancias** de otra entidad.	Un estudiante puede estar en varias asignaturas, y una asignatura tiene varios estudiantes.

Conceptos importantes en las relaciones

Propiedad dueña de la relación

En una relación, JPA necesita saber **qué entidad controla la relación** en la base de datos (es decir, cuál contiene la clave foránea).

Esto se indica con los atributos:

- `mappedBy` → define el lado **no propietario**.
- El lado **propietario** es el que contiene la clave foránea.

Cardinalidad

Define cuántas instancias de una entidad se pueden asociar con otra.
Ejemplo:

- En una relación `@OneToMany`, una entidad “A” puede tener muchas “B”.
- En una relación `@ManyToOne`, muchas “B” pertenecen a una “A”.

Direccionalidad

Las relaciones pueden ser:

- **Unidireccionales**: solo una entidad conoce la relación.
- **Bidireccionales**: ambas entidades conocen la relación (por ejemplo, un cliente conoce sus pedidos y un pedido conoce su cliente).

Cascada y fetch

- `cascade` → determina si las operaciones (persist, remove, etc.) se aplican también a las entidades relacionadas.
- `fetch` → indica si la relación se carga automáticamente (`EAGER`) o bajo demanda (`LAZY`).

Tipo	Clave foránea	Dirección típica	Anotaciones comunes
`@OneToOne`	En una de las tablas	Uni o bidireccional	`@OneToOne`, `@JoinColumn`
`@OneToMany` / `@ManyToOne`	En el lado “muchos”	Bidireccional	`@OneToMany(mappedBy="...")`, `@ManyToOne`
`@ManyToMany`	Tabla intermedia	Uni o bidireccional	`@ManyToMany`, `@JoinTable`

Relación One To Many (Uno a muchos)

Concepto

Una relación **uno a muchos** significa que **una entidad puede estar relacionada con varias instancias de otra entidad**, pero cada una de esas instancias pertenece solo a una de la primera.

En el caso de los alumnos:

- **Un alumno** puede tener **muchas notas**.
- Cada **nota** pertenece a **un solo alumno**.

En el modelo de base de datos:

- La tabla `notas` tiene una **clave foránea** (`id_alumno`) que apunta a la tabla `alumnos`.

Esquema conceptual

```
Alumno (1) ——————————< (N) Nota
```

- **Alumno** → entidad principal (lado “uno”)
- **Nota** → entidad dependiente (lado “muchos”)

Tablas en la base de datos

```
-- ====================================
-- Tabla principal: ALUMNOS
-- ====================================
CREATE TABLE alumnos (
   id_alumno INT AUTO_INCREMENT PRIMARY KEY,  -- Identificador único del alumno
   nombre VARCHAR(50) NOT NULL,               -- Nombre del alumno
   apellidos VARCHAR(100) NOT NULL            -- Apellidos del alumno
);


-- ====================================
-- Tabla dependiente: NOTAS
-- ====================================
CREATE TABLE notas (
   id_nota INT AUTO_INCREMENT PRIMARY KEY,    -- Identificador único de la nota
```

```
    asignatura VARCHAR(50) NOT NULL,     -- Nombre de la asignatura
    calificacion DECIMAL(4,2) NOT NULL,  -- Nota numérica (ej. 8.50)
    id_alumno INT NOT NULL,              -- Clave foránea: referencia al alumno
    CONSTRAINT fk_alumno                 -- Definición de la clave foránea
        FOREIGN KEY (id_alumno)
        REFERENCES alumnos(id_alumno)
        ON DELETE CASCADE                -- Si se borra un alumno, se borran sus notas
        ON UPDATE CASCADE                -- Si cambia el ID del alumno, se actualiza en notas
);

-- Insertamos dos alumnos
INSERT INTO alumnos (nombre, apellidos)
VALUES ('Laura', 'Pérez García'),
       ('Marcos', 'Ruiz Torres');

-- Insertamos notas para Laura (id_alumno = 1)
INSERT INTO notas (asignatura, calificacion, id_alumno)
VALUES ('Matemáticas', 8.5, 1),
       ('Historia', 9.0, 1);

-- Insertamos notas para Marcos (id_alumno = 2)
INSERT INTO notas (asignatura, calificacion, id_alumno)
VALUES ('Lengua', 7.5, 2);
```

Tabla: alumnos

id_alumno	nombre	apellidos
1	Laura	Pérez García
2	Marcos	Ruiz Torres

Tabla: notas

id_nota	asignatura	calificacion	id_alumno
1	Matemáticas	8.5	1
2	Historia	9.0	1
3	Lengua	7.5	2

Implementación en JPA

Clase `Alumno.java`

```
import jakarta.persistence.*;
import java.util.List;

@Entity
@Table(name = "alumnos")
public class Alumno {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id_alumno;

    private String nombre;
    private String apellidos;

    // Relación uno a muchos: un alumno tiene muchas notas
    @OneToMany(mappedBy = "alumno", cascade = CascadeType.ALL, fetch
= FetchType.LAZY)
    private List<Nota> notas;

    // Constructor vacío obligatorio
    public Alumno() {}

    // Constructor con parámetros
    public Alumno(String nombre, String apellidos) {
        this.nombre = nombre;
        this.apellidos = apellidos;
    }

    // Getters y setters
    public Long getId_alumno() { return id_alumno; }
    public String getNombre() { return nombre; }
    public String getApellidos() { return apellidos; }
    public List<Nota> getNotas() { return notas; }

    public void setNotas(List<Nota> notas) {
        this.notas = notas;
    }
}
```

Clase Nota.java

```
import jakarta.persistence.*;

@Entity
@Table(name = "notas")
public class Nota {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id_nota;

    private String asignatura;
    private double calificacion;

    // Relación muchos a uno: muchas notas pertenecen a un alumno
    @ManyToOne
    @JoinColumn(name = "id_alumno") // Clave foránea en la tabla
notas
    private Alumno alumno;

    public Nota() {}

    public Nota(String asignatura, double calificacion, Alumno
alumno) {
        this.asignatura = asignatura;
        this.calificacion = calificacion;
        this.alumno = alumno;
    }

    // Getters y setters
    public Long getId_nota() { return id_nota; }
    public String getAsignatura() { return asignatura; }
    public double getCalificacion() { return calificacion; }
    public Alumno getAlumno() { return alumno; }

    public void setAlumno(Alumno alumno) {
        this.alumno = alumno;
    }
}
```

Explicación de las anotaciones

Anotación	Se usa en	Significado
`@OneToMany(mappedBy="alumno")`	En `Alumno`	Indica que un alumno puede tener muchas notas. `mappedBy` señala el atributo que mantiene la relación en la clase `Nota`.
`@ManyToOne`	En `Nota`	Indica que muchas notas pertenecen a un solo alumno.
`@JoinColumn(name="id_alumno")`	En `Nota`	Define el nombre de la columna que actúa como clave foránea.
`cascade = CascadeType.ALL`	En `Alumno`	Permite que al guardar o eliminar un alumno, también se guarden o eliminen sus notas.
`fetch = FetchType.LAZY`	En `Alumno`	Carga las notas solo cuando se necesitan (optimiza el rendimiento).

Ejemplo de uso en código

```
// Crear un alumno
Alumno alumno = new Alumno("Laura", "Pérez García");

// Crear sus notas
Nota n1 = new Nota("Matemáticas", 8.5, alumno);
Nota n2 = new Nota("Historia", 9.0, alumno);

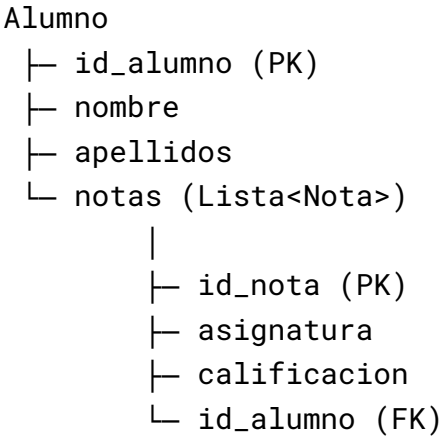
// Asignar notas al alumno
alumno.setNotas(List.of(n1, n2));

// Guardar en la base de datos
alumnoRepository.save(alumno);
```

Esto insertará:

- Una fila en `alumnos`.
- Dos filas en `notas`, con `id_alumno` apuntando al alumno creado.

Resumen gráfico



Relación Many To One (Muchos a uno)

Concepto

Una relación **muchos a uno** indica que **varias instancias de una entidad** están asociadas a **una única instancia de otra entidad**.

En nuestro ejemplo:

- **Muchas notas** pueden pertenecer al **mismo alumno**.
- Cada **nota** se relaciona con **un único alumno**.

Esquema conceptual

```
Nota (N) ————————————> (1) Alumno
```

- Cada **nota** tiene un **alumno** asignado.
- Un **alumno** puede tener varias notas.

Estructura de las tablas

Estas tablas son las mismas que definimos antes, pero ahora analizamos la relación desde el punto de vista del lado **"muchos"** (`notas`):

```
CREATE TABLE alumnos (
    id_alumno INT AUTO_INCREMENT PRIMARY KEY,
    nombre VARCHAR(50) NOT NULL,
    apellidos VARCHAR(100) NOT NULL
);

CREATE TABLE notas (
    id_nota INT AUTO_INCREMENT PRIMARY KEY,
    asignatura VARCHAR(50) NOT NULL,
    calificacion DECIMAL(4,2) NOT NULL,
    id_alumno INT NOT NULL,
    CONSTRAINT fk_alumno
        FOREIGN KEY (id_alumno)
        REFERENCES alumnos(id_alumno)
        ON DELETE CASCADE
        ON UPDATE CASCADE
);
```

En esta relación:

- `id_alumno` en la tabla `notas` es la **clave foránea**.
- Cada fila en `notas` **apunta** a una fila de `alumnos`.

Implementación en JPA

Clase `Nota.java`

```
import jakarta.persistence.*;

@Entity
@Table(name = "notas")
public class Nota {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id_nota;

    private String asignatura;
    private double calificacion;

    // Relación Muchos a Uno: muchas notas pertenecen a un alumno
    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "id_alumno", nullable = false)
    private Alumno alumno;

    public Nota() {}

    public Nota(String asignatura, double calificacion, Alumno
alumno) {
        this.asignatura = asignatura;
        this.calificacion = calificacion;
        this.alumno = alumno;
    }

    // Getters y setters
    public Long getId_nota() { return id_nota; }
    public String getAsignatura() { return asignatura; }
    public double getCalificacion() { return calificacion; }
```

```
    public Alumno getAlumno() { return alumno; }

    public void setAlumno(Alumno alumno) {
        this.alumno = alumno;
    }
}
```

Clase `Alumno.java` (opcional en unidireccional)

Si solo queremos la relación desde el lado de las **notas**, no hace falta modificar `Alumno`. Pero si queremos hacerla **bidireccional**, se añade en `Alumno`:

```
@OneToMany(mappedBy = "alumno", cascade = CascadeType.ALL)
private List<Nota> notas;
```

Explicación de las anotaciones

Anotación	Se usa en	Descripción
`@ManyToOne`	En `Nota`	Indica que muchas notas pertenecen a un solo alumno.
`@JoinColumn(name="id_alumno")`	En `Nota`	Define la columna de clave foránea que conecta con `Alumno`.
`fetch = FetchType.LAZY`	En `Nota`	Carga el alumno solo cuando se necesita (optimiza rendimiento).
`nullable = false`	En `@JoinColumn`	Evita que una nota quede sin alumno asociado.

Ejemplo de uso en código

```
// Creamos un alumno
Alumno alumno = new Alumno("María", "López Díaz");

// Creamos varias notas asociadas a ese alumno
Nota n1 = new Nota("Matemáticas", 9.0, alumno);
Nota n2 = new Nota("Lengua", 8.5, alumno);
```

```
Nota n3 = new Nota("Historia", 7.5, alumno);

// Guardamos primero el alumno
alumnoRepository.save(alumno);

// Luego las notas
notaRepository.saveAll(List.of(n1, n2, n3));
```

Esto insertará:

- Una fila en `alumnos`.
- Tres filas en `notas` con el mismo `id_alumno`.

Resumen gráfico

```
NOTAS
 ├─ id_nota (PK)
 ├─ asignatura
 ├─ calificacion
 └─ id_alumno (FK) ─────► ALUMNOS.id_alumno
```

Relación One To One (Uno a uno)

Concepto

Una relación **uno a uno** significa que **una instancia de una entidad** está asociada con **una única instancia de otra entidad**, y viceversa.

En nuestro ejemplo:

- **Un alumno** tiene **un solo perfil**.
- **Un perfil** pertenece **a un solo alumno**.

Esquema conceptual

```
Alumno (1) ———————— (1) Perfil
```

- **Alumno**: contiene la información principal.
- **Perfil**: contiene datos adicionales del mismo alumno.

Estructura de las tablas

```
-- ======================================
-- Tabla principal: ALUMNOS
-- ======================================
CREATE TABLE alumnos (
    id_alumno INT AUTO_INCREMENT PRIMARY KEY,
    nombre VARCHAR(50) NOT NULL,
    apellidos VARCHAR(100) NOT NULL
);

-- ======================================
-- Tabla dependiente: PERFIL
-- ======================================
CREATE TABLE perfil (
    id_perfil INT AUTO_INCREMENT PRIMARY KEY,
    direccion VARCHAR(100),
    telefono VARCHAR(20),
    foto_url VARCHAR(200),
    id_alumno INT UNIQUE,  -- Cada perfil solo puede pertenecer a un
alumno
```

```
    CONSTRAINT fk_perfil_alumno
        FOREIGN KEY (id_alumno)
        REFERENCES alumnos(id_alumno)
        ON DELETE CASCADE
        ON UPDATE CASCADE
);
```

Importante:
El campo `id_alumno` en la tabla `perfil` tiene la restricción `UNIQUE`, lo que garantiza que **un mismo alumno no tenga más de un perfil**.

Implementación en JPA

Clase `Alumno.java`

```
import jakarta.persistence.*;

@Entity
@Table(name = "alumnos")
public class Alumno {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id_alumno;

    private String nombre;
    private String apellidos;

    // Relación uno a uno: un alumno tiene un perfil
    @OneToOne(mappedBy = "alumno", cascade = CascadeType.ALL, fetch
= FetchType.LAZY)
    private Perfil perfil;

    public Alumno() {}

    public Alumno(String nombre, String apellidos) {
        this.nombre = nombre;
        this.apellidos = apellidos;
    }

    // Getters y setters
```

```
    public Long getId_alumno() { return id_alumno; }
    public String getNombre() { return nombre; }
    public String getApellidos() { return apellidos; }
    public Perfil getPerfil() { return perfil; }

    public void setPerfil(Perfil perfil) {
        this.perfil = perfil;
        perfil.setAlumno(this); // mantenemos sincronía
    }
}
```

Clase `Perfil.java`

```
import jakarta.persistence.*;

@Entity
@Table(name = "perfil")
public class Perfil {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id_perfil;

    private String direccion;
    private String telefono;
    private String foto_url;

    // Relación uno a uno: el perfil pertenece a un alumno
    @OneToOne
    @JoinColumn(name = "id_alumno", unique = true)
    private Alumno alumno;

    public Perfil() {}

    public Perfil(String direccion, String telefono, String
foto_url) {
        this.direccion = direccion;
        this.telefono = telefono;
        this.foto_url = foto_url;
    }
```

```
    // Getters y setters
    public Long getId_perfil() { return id_perfil; }
    public String getDireccion() { return direccion; }
    public String getTelefono() { return telefono; }
    public String getFoto_url() { return foto_url; }
    public Alumno getAlumno() { return alumno; }

    public void setAlumno(Alumno alumno) {
        this.alumno = alumno;
    }
}
```

Explicación de las anotaciones

Anotación	Se usa en	Significado
`@OneToOne`	En ambas clases	Define la relación uno a uno.
`@JoinColumn(name="id_alumno")`	En `Perfil`	Crea la clave foránea hacia `Alumno`.
`mappedBy="alumno"`	En `Alumno`	Indica que el dueño de la relación está en la clase `Perfil`.
`cascade = CascadeType.ALL`	En `Alumno`	Permite guardar/eliminar el perfil junto con el alumno.
`fetch = FetchType.LAZY`	En `Alumno`	Carga el perfil solo cuando se necesite.
`unique = true`	En `@JoinColumn`	Asegura que cada alumno tenga solo un perfil.

Ejemplo de uso en código

```
// Crear un alumno
Alumno alumno = new Alumno("Lucía", "Martínez Sánchez");

// Crear su perfil
Perfil perfil = new Perfil("Calle Mayor 25", "654321987",
"lucia.jpg");
```

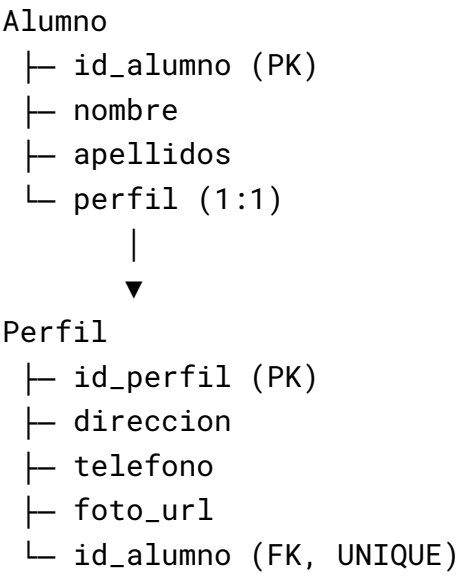
```
// Asociar ambos objetos
alumno.setPerfil(perfil);

// Guardar el alumno (y automáticamente el perfil)
alumnoRepository.save(alumno);
```

Esto insertará:

- Una fila en la tabla `alumnos`.
- Una fila en la tabla `perfil`, con `id_alumno` apuntando al alumno recién creado.

Resumen gráfico



Relación Many To Many (Muchos a muchos)

Concepto

Una relación **muchos a muchos** se da cuando **varias instancias de una entidad** pueden asociarse con **varias instancias de otra entidad**.
En las bases de datos relacionales, esto se resuelve mediante una **tabla intermedia** que contiene las claves foráneas de ambas tablas.

En nuestro ejemplo:

- Un **alumno** puede cursar **varias asignaturas**.
- Una **asignatura** puede tener **muchos alumnos**.

Esquema conceptual

```
Alumno (N) ——————< Alumno_Asignatura >—————— (N) Asignatura
```

Estructura de las tablas en MySQL

```
-- ======================================
-- Tabla ALUMNOS
-- ======================================
CREATE TABLE alumnos (
    id_alumno INT AUTO_INCREMENT PRIMARY KEY,
    nombre VARCHAR(50) NOT NULL,
    apellidos VARCHAR(100) NOT NULL
);

-- ======================================
-- Tabla ASIGNATURAS
-- ======================================
CREATE TABLE asignaturas (
    id_asignatura INT AUTO_INCREMENT PRIMARY KEY,
    nombre VARCHAR(100) NOT NULL,
    curso VARCHAR(50)
);

-- ======================================
-- Tabla intermedia ALUMNO_ASIGNATURA
-- ======================================
```

```
CREATE TABLE alumno_asignatura (
    id_alumno INT NOT NULL,
    id_asignatura INT NOT NULL,

    -- Clave primaria compuesta
    PRIMARY KEY (id_alumno, id_asignatura),

    -- Claves foráneas
    CONSTRAINT fk_alumno_as
        FOREIGN KEY (id_alumno)
        REFERENCES alumnos(id_alumno)
        ON DELETE CASCADE
        ON UPDATE CASCADE,

    CONSTRAINT fk_asignatura_al
        FOREIGN KEY (id_asignatura)
        REFERENCES asignaturas(id_asignatura)
        ON DELETE CASCADE
        ON UPDATE CASCADE
);
```

Notas importantes:

- La tabla `alumno_asignatura` **no tiene un campo propio `id`**, sino una **clave primaria compuesta** formada por `id_alumno` y `id_asignatura`.
- Cada fila representa una **matrícula** (una relación entre un alumno y una asignatura).

Implementación en JPA

Clase `Alumno.java`

```
import jakarta.persistence.*;
import java.util.HashSet;
import java.util.Set;

@Entity
@Table(name = "alumnos")
public class Alumno {

    @Id
```

```
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id_alumno;

    private String nombre;
    private String apellidos;

    // Relación muchos a muchos con Asignatura
    @ManyToMany
    @JoinTable(
        name = "alumno_asignatura", // tabla intermedia
        joinColumns = @JoinColumn(name = "id_alumno"), // FK a esta
entidad
        inverseJoinColumns = @JoinColumn(name = "id_asignatura") //
FK a la otra entidad
    )
    private Set<Asignatura> asignaturas = new HashSet<>();

    public Alumno() {}

    public Alumno(String nombre, String apellidos) {
        this.nombre = nombre;
        this.apellidos = apellidos;
    }

    // Métodos para añadir o eliminar asignaturas
    public void matricularEn(Asignatura asignatura) {
        asignaturas.add(asignatura);
        asignatura.getAlumnos().add(this);
    }

    public void desmatricularDe(Asignatura asignatura) {
        asignaturas.remove(asignatura);
        asignatura.getAlumnos().remove(this);
    }

    // Getters y setters
    public Long getId_alumno() { return id_alumno; }
    public String getNombre() { return nombre; }
    public String getApellidos() { return apellidos; }
    public Set<Asignatura> getAsignaturas() { return asignaturas; }
}
```

Clase `Asignatura.java`

```
import jakarta.persistence.*;
import java.util.HashSet;
import java.util.Set;

@Entity
@Table(name = "asignaturas")
public class Asignatura {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id_asignatura;

    private String nombre;
    private String curso;

    // Lado inverso de la relación
    @ManyToMany(mappedBy = "asignaturas")
    private Set<Alumno> alumnos = new HashSet<>();

    public Asignatura() {}

    public Asignatura(String nombre, String curso) {
        this.nombre = nombre;
        this.curso = curso;
    }

    // Getters y setters
    public Long getId_asignatura() { return id_asignatura; }
    public String getNombre() { return nombre; }
    public String getCurso() { return curso; }
    public Set<Alumno> getAlumnos() { return alumnos; }
}
```

Explicación de las anotaciones

Anotación	Se usa en	Descripción
`@ManyToMany`	En ambas clases	Define la relación muchos a muchos.
`@JoinTable`	En `Alumno`	Indica el nombre de la tabla intermedia y las columnas que enlazan ambas entidades.
`mappedBy="asignaturas"`	En `Asignatura`	Indica que el lado propietario es el `Alumno`.
`Set<Entidad>`	Se recomienda Set en lugar de List	Evita duplicados y simplifica la comparación de entidades.

Ejemplo de uso en código

```
// Crear asignaturas
Asignatura mat = new Asignatura("Matemáticas", "1º ESO");
Asignatura len = new Asignatura("Lengua", "1º ESO");

// Crear alumnos
Alumno ana = new Alumno("Ana", "García López");
Alumno juan = new Alumno("Juan", "Martín Ruiz");

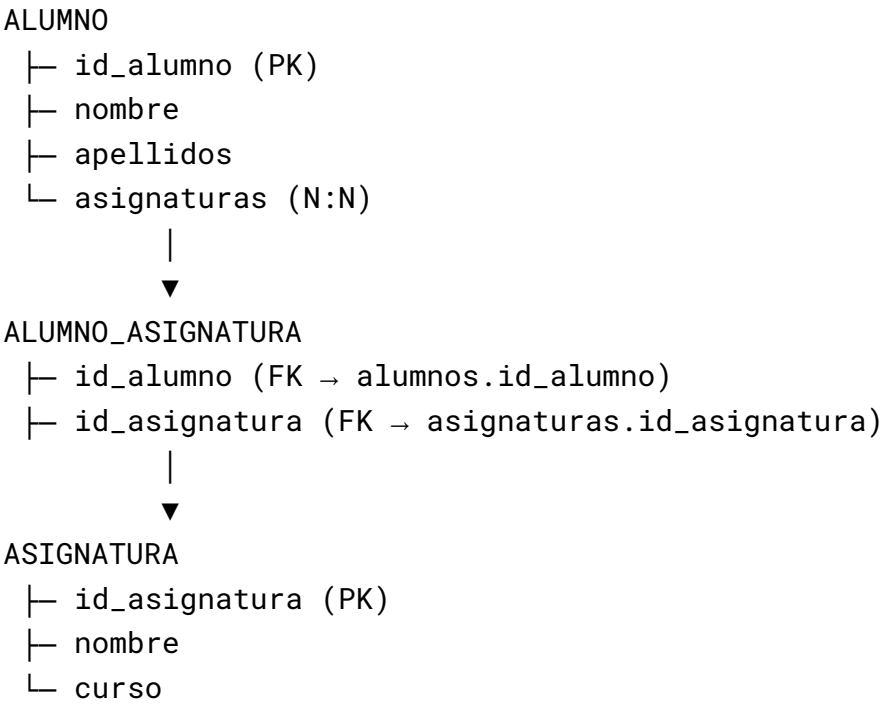
// Matricular alumnos
ana.matricularEn(mat);
ana.matricularEn(len);
juan.matricularEn(mat);

// Guardar en la base de datos
asignaturaRepository.saveAll(List.of(mat, len));
alumnoRepository.saveAll(List.of(ana, juan));
```

Esto creará:

- 2 filas en `asignaturas`
- 2 filas en `alumnos`
- 3 filas en `alumno_asignatura` (las relaciones alumno–asignatura)

Resumen gráfico



RESUMEN COMPARATIVO DE RELACIONES EN JPA

Tipo de relación	Descripción	Estructura SQL (resumen)	Anotaciones principales	Dirección habitual
@OneToOne	Una entidad está asociada con una única instancia de otra entidad.	Una tabla tiene una **clave foránea única** que apunta a otra tabla. `perfil.id_alumno UNIQUE REFERENCES alumnos(id_alumno)`	`@OneToOne`, `@JoinColumn`, `mappedBy`	Uni o bi-direccional
@OneToMany	Una entidad tiene varias instancias de otra entidad asociadas.	La tabla del lado “muchos” tiene una **clave foránea** que apunta a la del lado “uno”. `notas.id_alumno REFERENCES alumnos(id_alumno)`	`@OneToMany(mappedBy="...")`, `@ManyToOne`, `@JoinColumn`	Bidireccional (frecuente)
@ManyToOne	Muchas instancias de una entidad pertenecen a una única instancia de otra.	La tabla del lado “muchos” contiene la **clave foránea** hacia la tabla “uno”.	`@ManyToOne`, `@JoinColumn`	Unidireccional o bidireccional
@ManyToMany	Varias entidades están relacionadas con varias de otra entidad.	Se crea una **tabla intermedia** con dos claves foráneas. `alumno_asignatura(id_alumno, id_asignatura)`	`@ManyToMany`, `@JoinTable`, `mappedBy`	Bidireccional (frecuente)

Ejemplo resumen de cada caso

@OneToOne — Alumno ↔ Perfil

```
CREATE TABLE alumnos (
  id_alumno INT AUTO_INCREMENT PRIMARY KEY,
  nombre VARCHAR(50)
);

CREATE TABLE perfil (
  id_perfil INT AUTO_INCREMENT PRIMARY KEY,
  direccion VARCHAR(100),
  id_alumno INT UNIQUE,
  FOREIGN KEY (id_alumno) REFERENCES alumnos(id_alumno)
);
```

Claves JPA:

```
@OneToOne(mappedBy="alumno", cascade=CascadeType.ALL)
private Perfil perfil;

@OneToOne
@JoinColumn(name="id_alumno", unique=true)
private Alumno alumno;
```

@OneToMany / @ManyToOne — Alumno ↔ Notas

```
CREATE TABLE alumnos (
  id_alumno INT AUTO_INCREMENT PRIMARY KEY,
  nombre VARCHAR(50)
);

CREATE TABLE notas (
  id_nota INT AUTO_INCREMENT PRIMARY KEY,
  asignatura VARCHAR(50),
  calificacion DECIMAL(4,2),
  id_alumno INT,
  FOREIGN KEY (id_alumno) REFERENCES alumnos(id_alumno)
);
```

Claves JPA:

```
@OneToMany(mappedBy="alumno")
private List<Nota> notas;

@ManyToOne
@JoinColumn(name="id_alumno")
private Alumno alumno;
```

@ManyToMany — Alumno ↔ Asignatura

```
CREATE TABLE alumnos (
  id_alumno INT AUTO_INCREMENT PRIMARY KEY,
  nombre VARCHAR(50)
);

CREATE TABLE asignaturas (
  id_asignatura INT AUTO_INCREMENT PRIMARY KEY,
  nombre VARCHAR(100)
);

CREATE TABLE alumno_asignatura (
  id_alumno INT,
  id_asignatura INT,
  PRIMARY KEY (id_alumno, id_asignatura),
  FOREIGN KEY (id_alumno) REFERENCES alumnos(id_alumno),
  FOREIGN KEY (id_asignatura) REFERENCES asignaturas(id_asignatura)
);
```

Claves JPA:

```
@ManyToMany
@JoinTable(
  name="alumno_asignatura",
  joinColumns=@JoinColumn(name="id_alumno"),
  inverseJoinColumns=@JoinColumn(name="id_asignatura")
)
```

```
private Set<Asignatura> asignaturas;

@ManyToMany(mappedBy="asignaturas")
private Set<Alumno> alumnos;
```

Detalles clave a recordar

Concepto	Descripción
Lado propietario	Es el que contiene la clave foránea o define `@JoinTable`.
`mappedBy`	Se usa en el lado **no propietario** para indicar el atributo que mantiene la relación.
Cascada (`cascade`)	Propaga operaciones como `PERSIST`, `REMOVE` o `MERGE` a las entidades relacionadas.
Carga (`fetch`)	`EAGER`: carga inmediata. `LAZY`: carga bajo demanda (recomendado por rendimiento).
Colección recomendada	En `@ManyToMany` o `@OneToMany` se prefiere `Set` para evitar duplicados.

Resumen global

```
@OneToOne        Alumno ———————— Perfil
@OneToMany       Alumno ——<———— Nota
@ManyToOne       Nota ——>———————— Alumno
@ManyToMany      Alumno ——<————>—— Asignatura
```